darkmpc, Milestone 4b walkthrough

A shuffle that cannot be tampered with

Tave La Rocque

4 November 2026

Milestone 4b: a shuffle that cannot be tampered with

The attack

The sort applies secret permutations by shuffling and then opening. The servers shuffle the secret permutation ρ together with the data, using a random π that no single server knows, and then open the shuffled ρ. The opened value is $\rho \circ \pi^{-1}$, which is uniformly random and so reveals nothing. That reasoning assumes everyone follows the protocol.

Asharov et al. (CCS 2022, Section 5) show what a cheating server can do instead. A server in the pair applying one piece of π can add chosen errors to chosen coordinates of what it reshares. Suppose it wants to test whether $\rho(3) = \rho(2) + 1$. It adds errors that turn the opened vector back into a valid permutation exactly when the guess is right. Then it watches whether the opening succeeds. One bit of the secret leaks through success or abort: a *selective failure*.

The product checks of 4a cannot see this, because a shuffle contains no multiplications. That is why 4b exists.

The fix

Every value that goes through a shuffle carries a MAC. The servers draw a fresh secret random key α and compute $m = \alpha \cdot x$ for every value x, with one multiplication each. They shuffle values and MACs with the same permutation. Then, before anything is opened, they check a random linear combination: with public coefficients β_k drawn after the shuffle is fixed,

$$u = \sum_k \beta_k \, y_k, \qquad v = \sum_k \beta_k \, m_k, \qquad \text{check } \alpha u - v = 0.$$

The check is one more multiplication and one checked opening. If a cheater added error e_k to value k and f_k to its MAC, the opened value is

$$\alpha u - v = \alpha \sum_k \beta_k e_k - \sum_k \beta_k f_k,$$

because the honest parts cancel exactly: $\alpha \sum \beta_k x_{\pi(k)} = \sum \beta_k \, \alpha x_{\pi(k)}$. To pass, the cheater needs $\alpha \sum \beta_k e_k = \sum \beta_k f_k$. That requires knowing α, which it never sees, and the β's, which do not exist yet when it chooses its errors. Errors in the MAC multiplications themselves fall under the same check. This is the structure of Asharov et al.'s Protocol 5.4.

Why 128 bits

Their analysis works over a field. In $\mathbb{Z}_{2^{64}}$ it fails. An error of $e = 2^{63}$ gives $\alpha e \equiv 0$ whenever α is even, so the cheater passes half the time without knowing anything. The standard remedy comes from SPDZ2k (Cramer, Damgård, Escudero, Scholl and Xing, CRYPTO 2018, Section 3): do the MAC arithmetic modulo 2^{k+s}, where k is the data size and s is the security margin. Their Claim 1 bounds a single check's failure to catch an error that is nonzero modulo 2^k at 2^{-s}. Their Theorem 1 bounds a batch check with random coefficients at $2^{-s+\log(s+1)}$. With $k = s = 64$ everything fits in a Rust `u128`, which is about 2^{-58} per check.

Two facts make 128 bits cheap here.

Lifting is free. A replicated sharing of x modulo 2^{64}, read as numbers modulo 2^{128}, is a sharing of $x' = x + c \cdot 2^{64}$ for some small c. We never need to know c, because $x' \equiv x \pmod{2^{64}}$.

Reducing back is free. Taking each share modulo 2^{64} preserves addition and multiplication (a ring homomorphism). So after the verified shuffle in $\mathbb{Z}_{2^{128}}$, the reduced shares are exactly shares of the shuffled 64-bit values.

What is mine and what is published. The shuffle is Asharov et al.'s. The MAC check and its bound in rings are SPDZ2k's. SPDZ2k was designed for a dishonest majority with additive sharing. Combining it with the honest-majority replicated shuffle is my construction, and it needs its own written proof in the final report. So does the extension of the product checks to $\mathbb{Z}_{2^{64}}$ from 4a. Both arguments are short, and the write-up will set them out step by step. Until an expert has read them, the README calls them what they are: arguments by the author, not peer-reviewed results.

Testing

Two tests cover the new code.

- **Correctness.** A verified shuffle of random permutations on random data, with two columns, returns exactly the permuted values on all three servers. The check passes.
- **Tampering.** Party 1 adds 1 to one value while resharing. Every honest server aborts, and nothing is opened.

The planted bug `plant-skip-shuffle-check` switches the MAC check off. The tampering test then fails, because no server aborts: nothing else in the system notices. That is the selective-failure gap of Section 5, shown concretely. With it, `check_planted.sh` runs 20 planted bugs, all caught.

Cost

Orders	Mode	Rounds	Data per server	Time
1,000	malicious (4a only), TLS, simulated LAN	520	144 MB	1.35 s
1,000	malicious (4a + 4b), TLS, simulated LAN	595	156 MB	1.39 s
10,000	malicious (4a + 4b), in-process	774	1.70 GB	9.78 s

Verifying the shuffles adds about 8% data and 75 rounds at 1,000 orders. The cost is small because the shuffles were never the expensive part. The product checks are, and that is where the next optimisation goes: comparing hashes of the servers' views instead of sending second copies.

Security of a whole auction

Each product batch is checked to 2^{-56} (FLNW's bound). Losing 6 bits for 64 gates per AND word and $\log_2$ of the number of batches leaves at least 45.7 bits at 1,000 orders and 44.6 at 10,000, as computed by each run. The 15 shuffle checks per auction (14 sort bits plus the unsort) each add at most about 2^{-58}, which does not change the total at this precision. A union bound over everything stays above 44 bits for these batch sizes.

Where this leaves Milestone 4

Every part of the pipeline is now covered in malicious mode:

- trader inputs (validated on shares),
- multiplications and AND gates (FLNW cut-and-choose),
- shuffles (MACs modulo 2^{128}),
- openings (both copies compared),
- the verdict (unanimous abort).

The honest description is: **actively secure with abort against one corrupt server, under the stated assumptions, with the two ring extensions argued by the author.** The fair benchmark against the 2018 paper needs real machines next.

Optimisation: checking second copies by checksum

A checked opening used to send every value twice: once from the server that is supposed to send it, and once from the other server that holds the same summand. The

receiver compared the two. Now the second copy is never sent. The receiver and the other holder each fold their version into a running checksum,

$$\sum_k r_k v_k \bmod 2^{128},$$

with 64-bit keys r_k drawn from the pseudorandom stream only those two servers share.

Why the checksum cannot be fooled. The only server that could send a wrong value is the third one, and it never sees the key. Suppose it changes a value by an error e with $0 < |e| < 2^{64}$. Then e has 2-adic valuation $v < 64$. Two different keys below 2^{64} cannot give the same $re \bmod 2^{128}$, because their difference would need valuation at least $128 - v > 64$. So a wrong value survives with probability at most 2^{-64}. The sums are cumulative, so an earlier mismatch can never be cancelled later. The planted wrong-copy cheat is still caught.

The first version cost rounds. It compared checksums in an extra round before each verdict, because a server must not announce "pass" before it has checked its copies. Measured on a Mac with a core per server, it cut data by 38% and made the LAN faster, but it made the WAN slower:

1,000 orders, malicious, simulated network	Full copies	Checksum, extra round
Data per server	155.6 MB	97.0 MB
Rounds	595	647
LAN	1.126 s	0.918 s
LAN, 10,000 orders	7.77 s	7.33 s
WAN (40 ms round trip)	15.75 s	16.86 s

The WAN loss is exactly what the extra rounds predict: 52 more rounds at 20 ms one-way latency is 1.04 s, against 1.11 s measured. That is a nice confirmation that the latency model is doing its job.

A second mistake, and a measurement lesson. The next version sent the checksum back in the same round as the values, and the round counter dropped to 595. It was still slower on the Mac:

	Full copies	Piggybacked (flawed)
LAN	1.126 s	1.251 s
LAN, 10,000 orders	7.77 s	13.10 s
WAN	15.75 s	18.65 s

The cause was a hidden round. Each server computed its checksum only after it had received and hashed the incoming values. Its neighbour was waiting for that checksum, so every opening had become two dependent message hops while the counter recorded one. A round counter only means something if it counts dependent hops, and I had let it count calls. The fix is that a server's own copies are already in its hands. It folds them into its checksum before receiving anything and sends the checksum alongside its values, so each opening is one hop again.

The other lesson is about method. The same “full copies” code took 0.877 s on one run and 1.126 s on another, a 28% spread from noise alone, so single runs cannot decide a 10 to 20% question. `scripts/ab_bench.sh REF` now builds a git ref in a temporary worktree next to the working tree and alternates the two versions run by run, so slow periods hit both equally. It reports the median of each, and every run is still checked against the reference. Run against itself with two repetitions it shows differences of up to 7%, which gives a feel for the noise floor. The decision on this optimisation waits for a median-of-5 comparison on a machine with a core per server.

The decision. Median of 5 interleaved runs each, on a Mac with a core per server, simulated networks, 1,000 orders unless noted:

Config	Full copies	Checksum (fixed)	Change
LAN	1.109 s	1.039 s	−6% (within noise)
LAN, 10,000 orders	6.835 s	7.940 s	+16%
WAN	15.588 s	15.666 s	+1% (within noise)

The 38% data saving does not turn into speed in these settings. On the simulated LAN the link was never the bottleneck, the CPU was, and the checksums add CPU work. On the WAN, time is dominated by rounds (595 × 20 ms ≈ 12 s), so saving 59 MB per server barely registers. So full copies remain the default: simpler, and faster where it matters here. The checksum path stays in the code behind `--checksum-copies`, tested in both modes, with the planted wrong-copy cheat caught in each, for links where bandwidth really is the limit. It is worth re-measuring on real machines.

The lesson worth keeping is that an optimisation that cuts a cost which is not the bottleneck buys nothing. Measuring before believing, with medians, saved me from shipping a slowdown as an improvement.